

Prompts utilizados

1. Validar la lógica del algoritmo
 - Objetivo: comprobar que la idea de construir el árbol a partir de una expresión postfija era correcta.
 - Resultado: se confirmó que el enfoque era válido: operandos como hojas y operadores como nodos que unen dos subárboles.
2. Implementar `build_expression_tree` usando la clase `Stack`
 - Objetivo: construir el árbol sin reemplazar la pila por una lista de Python.
 - Resultado: se integró el método usando tu clase `Stack` y se conectó con nodos de expresión.
3. Probar el método con expresiones postfijas
 - Objetivo: verificar que el método funcionara con ejemplos como:

```
○ a b +  
○ a b c + *  
○ 8 2 / 3 4 + *
```

- Resultado: el método construía correctamente el árbol y devolvía la raíz.
4. Añadir soporte para `$` como potencia
 - Objetivo: que la expresión pudiera usar `$` como operador de potencia.
 - Resultado: el método reconoció `$` correctamente.
 5. Crear un archivo aparte para pruebas
 - Objetivo: separar la prueba del código principal.
 - Resultado: se creó un archivo de prueba para ejecutar ejemplos fácilmente.
 6. Hacer que el programa pidiera la expresión por teclado
 - Objetivo: que el usuario pudiera escribir cualquier expresión postfija.
 - Resultado: el script solicitaba la entrada y construía el árbol dinámicamente.
 7. Mostrar recorridos y resultado numérico
 - Objetivo: que además de construir el árbol, se vieran:

```
○ preorden  
○ inorden  
○ postorden  
○ inorden con paréntesis  
○ resultado numérico
```

- Resultado: el programa quedó mucho más útil para verificar funcionalidad.
8. Mejorar el manejo de errores
 - Objetivo: que si la expresión era inválida, el programa no se rompiera.
 - Resultado: se agregó un manejo de errores con mensajes claros.

Aquí se enlistan los prompts que se utilizaron como apoyo